sktime 라이브러리

시계열 데이터 분석을 위한 sktime 라이브러리

TIME-SERIES

저자
Chunghun Ha

소속
Hongik University

공개
2023년 7월 28일

MODIFIED
2026년 3월 8일

sktime 라이브러리

sktime 라이브러리

sktime은 ***시계열 기계학습***을 위한 통합 Python 라이브러리

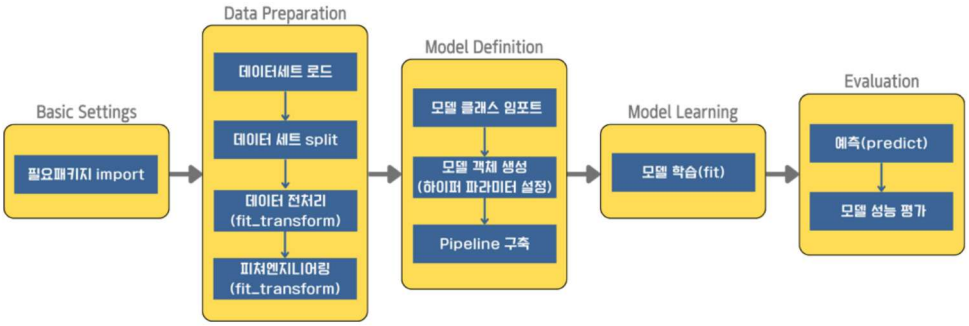
- sktime은 머신러닝 패키지로 가장 많이 사용하는 ***scikit-learn*** API를 시계열 작업으로 확장
  - scikit-learn 패키지는 인덱스에 시간을 사용할 수 없어 시계열 데이터를 다루기 어려움
  - sktime은 시계열 회귀, 예측 및 분류 작업을 효율적으로 해결하는 데 필요한 알고리즘 및 변환 도구를 제공
  - sktime은 ***scikit-learn의 프로세스(fit, transform, predict,...)를 그대로 차용***
- ***pandas***를 기본 데이터 구조로 사용(numpy도 사용 가능)
- sktime 설치
  - `uv add sktime`

`sktime`의 주요 기능

- Time Series Deterministic/Probabilistic Forecasting: 시계열 ***결정적/확률적 예측***
- Time Series Classification/Regression/Clustering: 시계열 ***분류/회귀/군집***
- Time Series Segmentation: 시계열 ***분할***

`sktime`의 적용 프로세스

- `sktime`은 scikit-learn과 유사한 프로세스를 따름



예제 데이터세트(`sktime`)

- `sktime`은 다양한 시계열 데이터세트를 제공
- 데이터세트 목록 확인: `sktime.datasets.__all__`
- 데이터세트 로드: `sktime.datasets.load_데이터세트명()`

```
import sktime.datasets as skdata

# 데이터세트 목록 가져오기
print(skdata.__all__)
```

```
['load_airline', 'load_arrow_head', 'load_gunpoint', 'load_basic_motions', 'load_osuleaf',
'load_italy_power_demand', 'load_japanese_vowels', 'load_plaid', 'load_longley', 'load_lynx',
'load_shampoo_sales', 'load_UCR_UEA_dataset', 'load_unit_test', 'load_uschange', 'load_PBS_dataset',
'load_japanese_vowels', 'load_gun_point_segmentation', 'load_electric_devices_segmentation', 'load_acsf1',
'load_macroeconomic', 'load_hierarchical_sales_toydata', 'generate_example_long_table',
'load_from_arff_to_dataframe', 'load_from_long_to_dataframe', 'load_from_tsfile',
'load_from_tsfile_to_dataframe', 'load_from_ucr_tsv_to_dataframe', 'make_multi_index_dataframe',
'load_tsf_to_dataframe', 'load_unit_test_tsf', 'load_solar', 'load_covid_3month', 'load_forecastingdata',
'load_m5', 'write_panel_to_tsfile', 'write_dataframe_to_tsfile', 'write_ndarray_to_tsfile',
'write_results_to_uea_format', 'write_tabular_transformation_to_arff', 'load_tecator', 'load_fpp3', '_load_fpp3',
'DATASET_NAMES_FPP3', 'BaseDataset', 'Airline', 'Longley', 'Lynx', 'Macroeconomic', 'PBS', 'ShampooSales',
'Solar', 'USChange', 'ForecastingData', 'M5Dataset', 'ACSF1', 'ArrowHead', 'BasicMotions', 'GunPoint',
'ItalyPowerDemand', 'JapaneseVowels', 'OSULeaf', 'PLAID', 'UCRUEADataset']
```

```
# load_shampoo_sales 데이터세트 로드
ts = skdata.load_shampoo_sales()

# 데이터세트 정보 출력
ts.info()
```

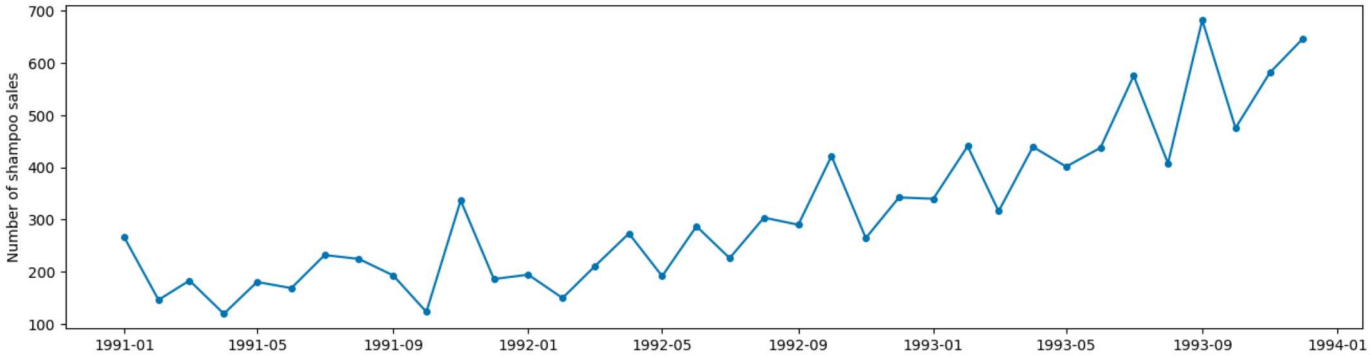
```
<class 'pandas.core.series.Series'>
PeriodIndex: 36 entries, 1991-01 to 1993-12
Freq: M
Series name: Number of shampoo sales
Non-Null Count  Dtype
--------------  -----
36 non-null     float64
dtypes: float64(1)
memory usage: 576.0 bytes
```

```
# 데이터세트 출력
ts.head()
```

```
Period
1991-01    266.0
1991-02    145.9
1991-03    183.1
1991-04    119.3
1991-05    180.3
Freq: M, Name: Number of shampoo sales, dtype: float64
```

- `plot_series()` 함수를 사용하여 시계열 데이터 시각화 가능

```
from sktime.utils.plotting import plot_series

plot_series(ts)
```



시계열 예측 예제(`sktime`)

데이터세트 불러오기

```
import numpy as np
import pandas as pd
import sktime.datasets as skdata
from sktime.utils.plotting import plot_series

# airline 데이터세트 로드 -> Pandas Series 형태로 로드됨
ts = skdata.load_airline()

# 데이터세트 정보 출력
print(ts.info())
```

```
<class 'pandas.core.series.Series'>
PeriodIndex: 144 entries, 1949-01 to 1960-12
Freq: M
Series name: Number of airline passengers
Non-Null Count  Dtype
--------------  -----
144 non-null    float64
dtypes: float64(1)
memory usage: 2.2 KB
None
```

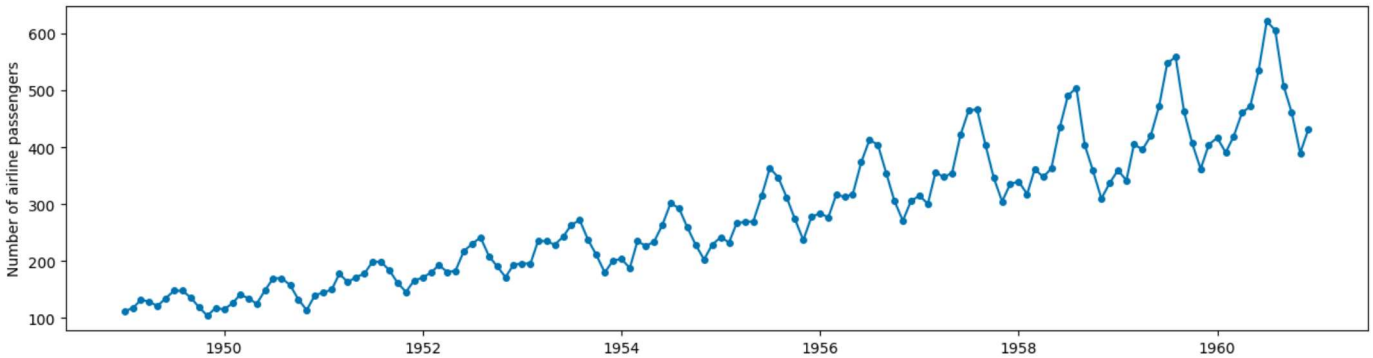
- 데이터 확인

```python
# 데이터 확인
print(ts.head())
print('-'*80)

# index 확인, PeriodIndex 형태로 되어 있음
print(ts.index)
print('-'*80)

# values 확인
print(ts.values)
```

```
Period
1949-01    112.0
1949-02    118.0
1949-03    132.0
1949-04    129.0
1949-05    121.0
Freq: M, Name: Number of airline passengers, dtype: float64
--------------------------------------------------------------------------------
PeriodIndex(['1949-01', '1949-02', '1949-03', '1949-04', '1949-05', '1949-06',
             '1949-07', '1949-08', '1949-09', '1949-10',
             ...
             '1960-03', '1960-04', '1960-05', '1960-06', '1960-07', '1960-08',
             '1960-09', '1960-10', '1960-11', '1960-12'],
            dtype='period[M]', name='Period', length=144)
--------------------------------------------------------------------------------
[112. 118. 132. 129. 121. 135. 148. 148. 136. 119. 104. 118. 115. 126.
 141. 135. 125. 149. 170. 170. 158. 133. 114. 140. 145. 150. 178. 163.
 172. 178. 199. 199. 184. 162. 146. 166. 171. 180. 193. 181. 183. 218.
 230. 242. 209. 191. 172. 194. 196. 196. 236. 235. 229. 243. 264. 272.
 237. 211. 180. 201. 204. 188. 235. 227. 234. 264. 302. 293. 259. 229.
 203. 229. 242. 233. 267. 269. 270. 315. 364. 347. 312. 274. 237. 278.
 284. 277. 317. 313. 318. 374. 413. 405. 355. 306. 271. 306. 315. 301.
 356. 348. 355. 422. 465. 467. 404. 347. 305. 336. 340. 318. 362. 348.
 363. 435. 491. 505. 404. 359. 310. 337. 360. 342. 406. 396. 420. 472.
 548. 559. 463. 407. 362. 405. 417. 391. 419. 461. 472. 535. 622. 606.
 508. 461. 390. 432.]
```

- 시계열 데이터 시각화

```python
plot_series(ts);
```

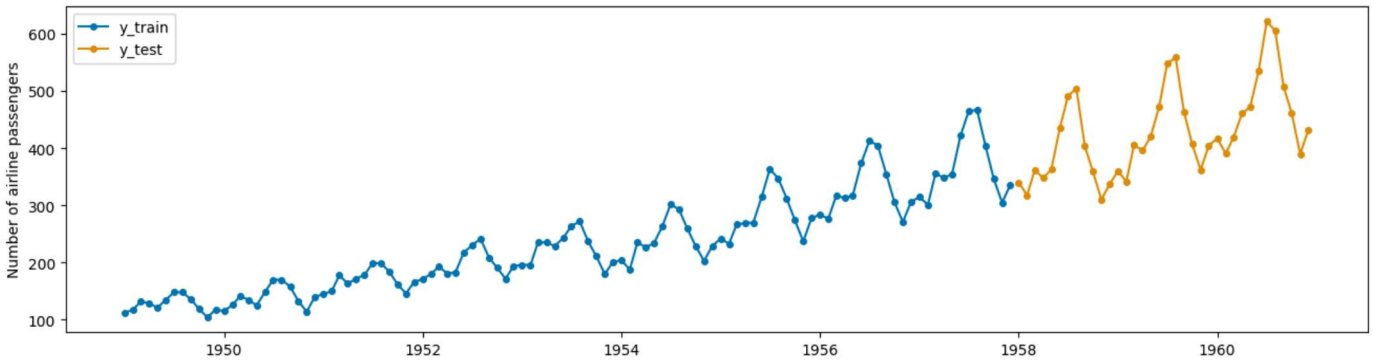


학습데이터와 평가데이터 분리

- `sklearn`과 동일하게 학습데이터와 평가데이터를 분리
  - `temporal_train_test_split()`을 사용 ← `sklearn`은 `train_test_split()`

```
# 시계열 데이터세트을 train, test로 나누는 함수
from sktime.forecasting.model_selection import temporal_train_test_split

# 36개월을 test 데이터세트으로 사용하고 나머지를 train 데이터세트으로 사용
y_train, y_test = temporal_train_test_split(ts, test_size=36)

# 데이터세트 개수 확인
print('#Train Data:', y_train.shape[0], ', #Test Data:', y_test.shape[0])
print('-'*80) # 구분선

# 시각화
plot_series(y_train, y_test, labels=["y_train", "y_test"]);
```

```
#Train Data: 108 , #Test Data: 36
--------------------------------------------------------------------------------
```



예측시평(forecasting horizon) 설정

- 시평(forecasting horizon)은 예측할 시점의 길이를 의미
- `forecasting horizon`을 설정하는 방법은 `forecasting` 모듈의 `ForecastingHorizon` 클래스를 사용
  - `is_relative`는 시평을 어떻게 적용할 것인지 여부
    - `True`: train data 이후에 상대적인 시평을 적용
    - `False`: 지정한 절대적인 시평을 사용

```
from sktime.forecasting.base import ForecastingHorizon   # 시평을 설정하는 함수

# 절대적 시평 설정 -> 시평을 평가데이터세트인 y_test와 동일 기간으로 설정한 경우
fh_absolute = ForecastingHorizon(y_test.index, is_relative=False)
fh_absolute
```

```
ForecastingHorizon(['1958-01', '1958-02', '1958-03', '1958-04', '1958-05', '1958-06',
             '1958-07', '1958-08', '1958-09', '1958-10', '1958-11', '1958-12',
             '1959-01', '1959-02', '1959-03', '1959-04', '1959-05', '1959-06',
             '1959-07', '1959-08', '1959-09', '1959-10', '1959-11', '1959-12',
             '1960-01', '1960-02', '1960-03', '1960-04', '1960-05', '1960-06',
```

```
 '1960-07', '1960-08', '1960-09', '1960-10', '1960-11', '1960-12'],
dtype='period[M]', is_relative=False)
```

```python
# 상대적 시평 설정 -> 시평을 마지막 train data 이후 10개로 설정한 경우
fh_relative = ForecastingHorizon(range(1,11), is_relative=True)
fh_relative
```

```
ForecastingHorizon(start=1, stop=11, step=1, is_relative=True)
```

예측모형 설정

- ***모든 예측모형은 클래스*** 이므로 이를 사용하기 위해서는 ***객체(object, instance)를 생성하여 사용***
  - ***객체 생성시 모형의 파라미터를 설정*** 하여 다양한 모형을 구성할 수 있음

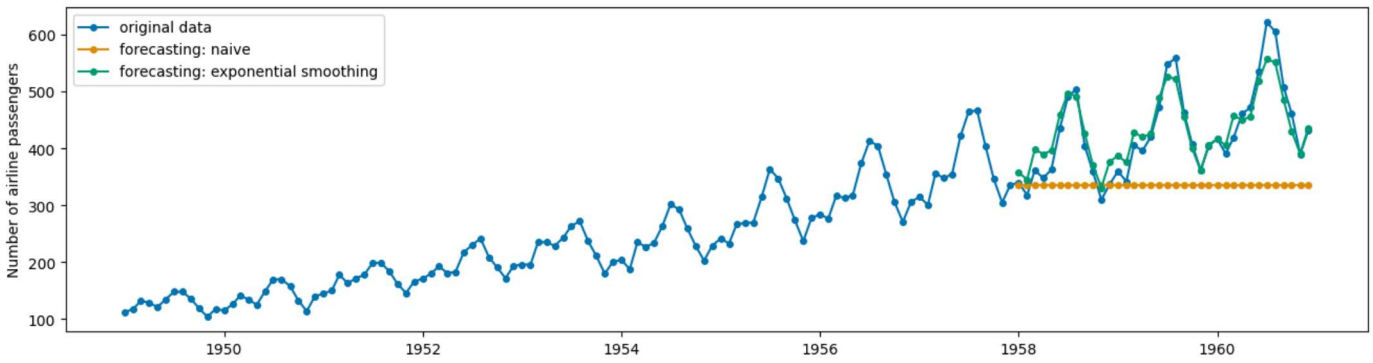
```python
from sktime.forecasting.exp_smoothing import ExponentialSmoothing  # 지수평활법 예측 알고리즘
from sktime.forecasting.naive import NaiveForecaster   # 단순 예측 알고리즘

# 예측 알고리즘 객체 생성

# 마지막 데이터를 예측값으로 사용하는 단순 예측 알고리즘
naive_forecaster = NaiveForecaster(strategy="last")

# 지수평활법 예측 알고리즘
exp_smoothing_forecaster = ExponentialSmoothing(trend="add", seasonal="additive", sp=12)
```

훈련과 예측

- 훈련은 훈련 데이터세트로 `fit()` 을 사용하여 모형의 계수를 추정
- 예측은 `fit()` 이 완료된 학습된 모형(추정한 계수)을 사용하여 평가 데이터세트에 대하여 `predict()` 로 추정값을 계산

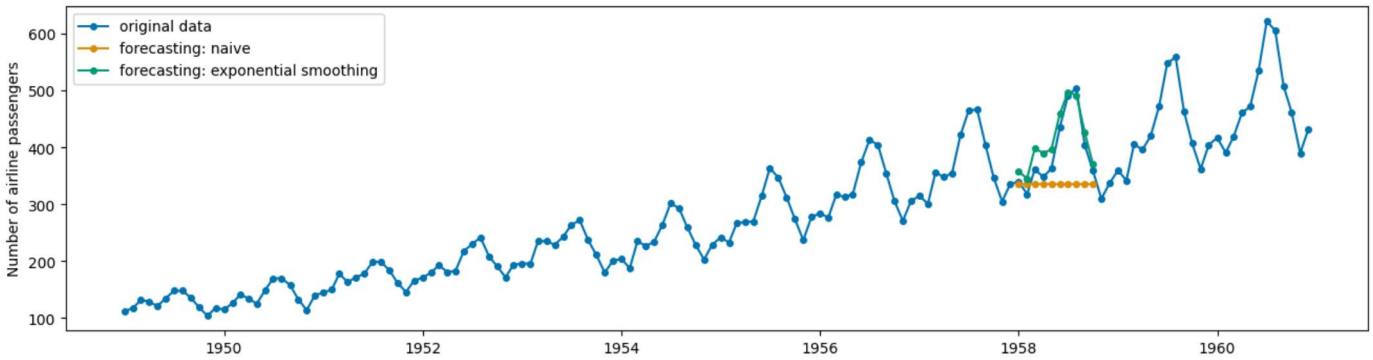
```python
# 훈련
# 단순 예측 알고리즘 학습(모형의 계수 추정)
naive_forecaster.fit(y_train)
# 지수평활법 예측 알고리즘 학습(모형의 계수 추정)
exp_smoothing_forecaster.fit(y_train)

# 예측
# 단순 예측 알고리즘을 사용하여 예측
y_pred_naive = naive_forecaster.predict(fh_absolute)
# 지수평활법 예측 알고리즘을 사용하여 예측
y_pred_exp_smoothing = exp_smoothing_forecaster.predict(fh_absolute)

# 시각화
plot_series(
    ts, y_pred_naive,  y_pred_exp_smoothing,
```

```
    labels=["original data", "forecasting: naive", "forecasting: exponential smoothing"]
    );
```



- 학습된 예측 모형에 상대적 시평을 적용한 경우

```
# 단순 예측 알고리즘을 사용하여 예측
y_pred_naive_relative = naive_forecaster.predict(fh_relative)
# 지수평활법 예측 알고리즘을 사용하여 예측
y_pred_exp_smoothing_relative = exp_smoothing_forecaster.predict(fh_relative)

# 시각화
plot_series(
    ts, y_pred_naive_relative,  y_pred_exp_smoothing_relative,
    labels=["original data", "forecasting: naive", "forecasting: exponential smoothing"]
    );
```



예측모형 성능 평가

- `sktime.performance_metrics.forecasting`에 다양한 평가지표를 제공

```
# MSE, MAPE 임포트
from sktime.performance_metrics.forecasting import mean_squared_error as mse
from sktime.performance_metrics.forecasting import mean_absolute_percentage_error as mape

# MSE, MAPE 계산
print("Naive         MSE:", mse(y_test, y_pred_naive))
print("ExpSmoothing  MSE:", mse(y_test, y_pred_exp_smoothing))
print("Naive        MAPE:", mape(y_test, y_pred_naive))
print("ExpSmoothing MAPE:", mape(y_test, y_pred_exp_smoothing))
```

```
Naive         MSE: 14674.555555555555
ExpSmoothing  MSE: 695.6840244843274
Naive        MAPE: 0.19886711926999853
ExpSmoothing MAPE: 0.05114155319583433
```

Copyright 2026, Chunghun Ha, Hongik University